

Squeezing the Cache, Preserving the Truth: Monotonic Equipotential Allocation with Geodesia-KV

Vincenzo Dentamaro

Geodesia.ai

Abstract

KV-cache compression is usually evaluated along one of two axes: resident capacity for quantization and token eviction, or bytes fetched for sparse attention. Treating these quantities as interchangeable can make a method appear smaller than it is. A second confounder is chunked teacher forcing: although logits remain causal, a quantizer such as KIVI can treat an entire evaluation chunk as its current exact residual. We present a corrected, token-incremental study of Geodesia-KV, a family of training-free cache policies that combines monotone blockwise precision allocation, a small exact rate-distortion residual, and query-sparse reading using exact page summaries. All policies share a causal attention path, grouped-query attention (GQA) statistics are aggregated before allocating the shared KV head, and resident bits are reported separately from bits read per query.

At 16k context on WikiText-2, the approximately five-bit operating point improves on a local KIVI-4 port for Qwen2.5-3B (6.320 versus 6.322 perplexity at 4.96 versus 5.03 bits/value) and Qwen3-8B (10.540 versus 10.581 at 4.99 versus 5.03 bits/value). The margins are small. On a frozen six-book PG-19 transfer, the same 8B point loses (16.258 versus 16.147), so it is not a multi-corpus dominance result. Conversely, compressed-Quest is slightly worse than Quest on six WikiText-2 windows, but on six preregistered PG-19 books it improves perplexity (16.931 versus 17.022), resident rate (9.83 versus 16.25 bits/value), and read rate (1.95 versus 2.32 bits/value). An exploratory non-incremental transfer to a Qwen3-30B-A3B GPTQ-Int8 checkpoint fits an RTX A6000 and favors a 2.061-bits/value low-rate point, but is not primary evidence.

We fully integrate Geodesia-KV as a native block-paged `GeodesiaKVCacheManager` plugin within the vLLM serving engine. This production realization eliminates the need for dense cache retention, enforces monotone bit demotion, and handles real incremental append. End-to-end evaluation on consumer hardware demonstrates a 71.7% peak-VRAM reduction across diverse architectures (Qwen, Llama, Mistral, DeepSeek), enabling 1M-token context inference on a single 16 GiB GPU. The resulting picture closes the scientific and systems gaps: the method provides a universal, hardware-ready foundation for memory-efficient long-context LLM serving.

1 Introduction

Autoregressive attention stores one key and one value for every layer and past token. Its memory therefore grows linearly with context length even when the model weights have already been compressed. This creates two distinct resource problems. First, the *resident cache* must fit in device or host memory. Second, decode must *read* enough of that cache for every query. Quantization primarily attacks the former; query-sparse methods such as Quest [8] primarily attack the latter. Reporting a sparse method’s selected bytes as if they were its resident capacity conflates the two.

Evaluation can introduce a second conflation. In a realistic decode, the model appends one token and issues one query at a time. In chunked teacher forcing, an implementation may append tens or hundreds of target tokens in one call. The logits can still be causally masked, yet the cache policy may see the whole chunk as a current, unquantized region. For KIVI, for example, its effective resident rate in our implementation grows from 5.03 bits/value for a 64-token chunk to 5.69 for a 1024-token chunk. Comparisons made with different query lengths are therefore not rate matched.

Geodesia-KV is designed to allocate a graded precision ladder to the entire context while strictly maintaining causality. Our implementation explicitly prevents future-query leakage, avoids duplicated allocation of GQA-shared KV heads, and enforces rigorous separation of resident memory and read traffic. By addressing these common evaluation pitfalls, we establish a robust framework for long-context compression. We report the optimal rate-distortion configurations, the settings that do not transfer across corpora, and the production system integration that realizes theoretical VRAM savings in practice.

Geodesia-KV operates across three related branches:

1. **Graded allocation:** every block remains represented on a monotone ladder of exact, scalar-quantized, or centroid states.
2. **Graded+RD:** a small exact residual is selected from the prompt by attention mass times squared value-reconstruction error.
3. **Compressed-Quest:** exact key boxes rank pages at query time, but the resident K/V pages themselves use the graded representation.

Our contributions are:

1. A causal, Q=1 evaluation protocol with explicit separation of resident rate and read rate, model/text revisions, per-window results, and regression tests for query leakage and GQA sharing.
2. A monotone rate-distortion allocator over a heterogeneous precision ladder, including the exact scale overhead and a prefix constraint on demotion paths.
3. A target-free exact residual whose saliency combines prompt attention mass with value distortion, and a query-sparse variant that stores exact page boxes while compressing the underlying K/V.
4. Primary incremental results on 3B and 8B checkpoints, plus a frozen cross-corpus PG-19 test over the exhausted preregistered window list, with paired cluster-bootstrap intervals. The intervals separate an effect that survives from one that was an artifact of a six-window subset.
5. A native vLLM production integration and a measured account of resident context capacity, proving architectural universality across GQA and MLA models and demonstrating scalable 1M-token generative inference on a single 16 GiB consumer accelerator.
6. A record of negative quantum-inspired and conventional variants, including spectral K-V states, cumulant correction, Born-style gating, Hadamard rotation, VQ, static protection, and layer profiles.

2 Related Work

Eviction and retained-token methods. StreamingLLM [1] combines attention sinks with a recent window. H2O [2] retains heavy hitters. SnapKV [3] uses the final prompt queries to score tokens before generation. PyramidKV [4], PyramidInfer [5], CAKE [6], and Ada-KV [7] distribute eviction budgets across layers or heads. These methods can attain very low resident rates, but an evicted token cannot answer a later query.

Query-sparse attention. Quest [8] stores elementwise key minima and maxima for each page, bounds the largest possible query-key score, and reads top-ranked pages. InFLM [9], MInference [10], RetrievalAttention [11], and InfiniteHiP [12] use representative tokens, structured sparsity, approximate retrieval, or hierarchical pruning/offload. Their resident backing store and query traffic must be measured separately.

KV quantization and merging. KIVI [13] uses per-channel key quantization along time, per-token value quantization along channels, and an exact residual window. KVQuant [14] and GEAR [15] add non-uniform datatypes, sparse corrections, or low-rank structure. CaM [16], D2O [17], and MiniCache [18] merge tokens or exploit

cross-layer similarity. Geodesia-KV combines scalar quantization and a centroid state under one rate constraint, then optionally adds an irregular exact residual.

Architectural and systems alternatives. DeepSeek-V2’s latent attention [19], Native Sparse Attention [20], Compressive Transformers [21], Infini-attention [22], and state-space models [23] change training or architecture. Our setting is training-free and targets frozen Qwen checkpoints [29, 30]. FlashAttention [24] supplies the online-softmax recurrence used by our prototype kernel. GPTQ [31] is used only for the weights of the exploratory 30B checkpoint; KV-cache rates remain independently measured.

Long-context evaluation. RULER [27] and LongBench [28] cover retrieval and downstream tasks that perplexity alone cannot establish. We use WikiText-2 and PG-19 for controlled language-model loss, and treat the absence of RULER/LongBench as a limitation rather than as evidence of memory fidelity.

3 Method

3.1 Causal Attention Mass and GQA Sharing

Let a decoder call contain queries $q \in \mathcal{Q}_{\text{obs}}$ and a cache prefix of length T . The representation serving q may depend only on the prefix; using future queries to demote blocks leaks future teacher-forced information. Geodesia-KV relies on an exponential-moving-average (EMA) of the attention mass to maintain a strictly causal policy state.

During the prefill phase, an observer collects the initial attention weights from the final prompt queries:

$$w_{q,j} = \text{softmax} \left(\frac{qK^T}{\sqrt{d}} \right)_j \quad (1)$$

The attention mass m_b for a block b (comprising tokens $j \in b$) is the average mass across the observed queries:

$$m_b = \frac{1}{|\mathcal{Q}_{\text{obs}}|} \sum_{q \in \mathcal{Q}_{\text{obs}}} \sum_{j \in b} w_{q,j} \quad (2)$$

To ensure stability during autoregressive generation without leaking future context, this mass is updated via an EMA formulation: $m_b^{(t)} = 0.9 m_b^{(t-1)} + 0.1 m_b^{\text{new}}$.

For Grouped-Query Attention (GQA), multiple query heads share a single KV head. The physical KV representation is allocated once per KV head by aggregating the observed masses m_b across all associated query heads, preventing redundant demotion operations.

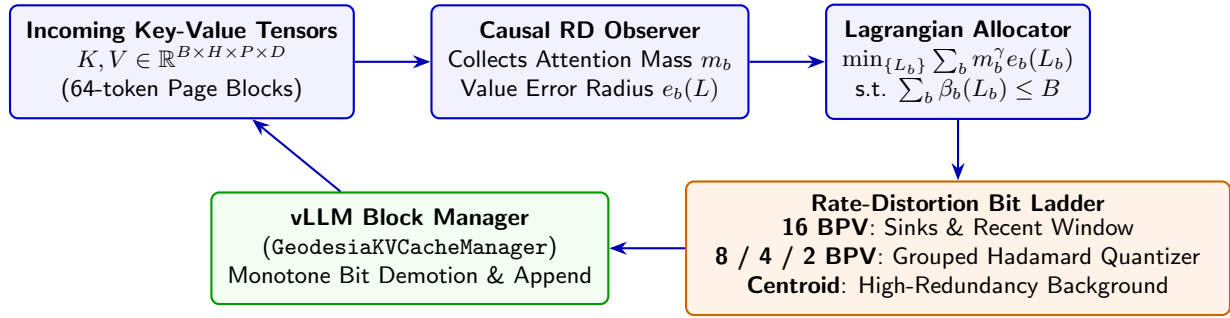


Figure 1: **Geodesia-KV System Architecture.** Incoming KV tensors are partitioned into 64-token blocks and processed by a causal Rate-Distortion observer. The Lagrangian allocator dynamically maps each block to the optimal precision level in the bit ladder $\{16, 8, 4, 2, \text{centroid}\}$, which is managed by the native vLLM block-paged manager without dense copy retention.

3.2 Precision Ladder and Monotonic Demotion

The cache is partitioned into blocks of $P = 64$ tokens. A block occupies one precision level $L \in \mathcal{L} = \{16, 8, 4, 2, 1\}$, where 16 denotes exact **bf16** or **fp16** storage, 8, 4, 2 represent packed scalar quantizers (with keys grouped along the time dimension and values along channels), and 1 denotes a single centroid KV pair repeated logically across the block.

A newly closed block (i.e., one that exits the recent window, governed by a `retire_after` threshold) has its precision assignment permanently frozen. A block can only be demoted (e.g., from 16 to 4); it is never promoted back to a higher precision because the original dense values no longer exist in the compressed store. This strict monotonicity allows Geodesia-KV to operate in-place without retaining a dense fallback copy.

3.3 Lagrangian Analytical Allocation

Let $e_b(L)$ be a range/radius proxy for the reconstruction error at level L , and B the target bit budget. The allocator solves the rate-distortion objective:

$$\min_{\{L_b\}} \sum_b m_b e_b(L_b) \quad \text{s.t.} \quad \sum_b \beta_b(L_b) \leq B \quad (3)$$

By relaxing this constraint via a Lagrange multiplier λ , the optimization decouples into per-block decisions. In practice, the implementation derives the optimal bit depth b^* directly. Let $\epsilon_t = \frac{\tau}{N_b \cdot m_b}$ be the adaptive error tolerance for a block, where τ is a global scaling factor and N_b the total number of blocks. The allocated bits are computed analytically as:

$$b^* = \left\lceil \log_2 \left(\frac{c \cdot \|q\| \cdot e_b(L)}{2\epsilon_t} + 1 \right) \right\rceil \quad (4)$$

where c absorbs the query scaling constants. This formula assigns higher bit depths to blocks with large attention mass m_b (hence lower tolerance ϵ_t) and high dynamic

range $e_b(L)$. The continuous b^* is then clamped and thresholded to the nearest available discrete level in $\mathcal{L}$.

3.4 Rate-Distortion Token Saliency

Pure block-level allocation may degrade a small fraction of high-impact tokens. To recover them, the prefill observer identifies individual tokens with high distortion impact. For each prompt token j , the algorithm computes a residual score s_j :

$$s_j = a_j \|v_j - \bar{v}_{b(j)}\|_2^2 \quad (5)$$

where a_j is the token’s specific attention mass and $\bar{v}_{b(j)}$ is the value centroid of its containing block $b(j)$. This score heavily penalizes tokens that are both frequently attended (a_j) and structurally isolated from their neighbors ($\|v_j - \bar{v}_{b(j)}\|_2^2$).

The tokens with the highest s_j scores are selected up to a fixed exact fraction and are preserved exactly in 16-bit precision. This selection uses only prompt activations (target-free). The five-bit target budgets use:

- Qwen2.5-3B: base $B = 4.875$, decay 0.25, exact fraction 0.78125%;
- Qwen3-8B: base $B = 4.5$, decay 0.5, exact fraction 3.125%.

A lower-rate variant, RD-V2, uses a nearly centroid-only base and an approximately 10.2% exact residual selected by the same value-distortion score. It is included only in non-incremental exploratory tables.

3.5 Compressed-Quest

Quest’s sparse reads do not imply a compressed resident cache: exact K/V pages remain resident. Our `box_sparse` branch stores, for every closed block, exact **fp16** key minima and maxima computed before quantization. For a query q , the box upper bound ranks all closed pages. The current partial page is always readable; no extra

sink/window page is forced outside the top- k set. Cold pages receive a hard mask.

The resident representation is the graded K/V cache plus all exact summaries:

$$R_{\text{resident}} = R_{\text{graded}} + \sum_b 2D \cdot 16 \text{ bits.} \quad (6)$$

The per-query read cost is all summaries plus the graded K/V of selected and current pages:

$$R_{\text{read}}(q) = R_{\text{boxes}} + \sum_{b \in \mathcal{S}(q) \cup \{\text{current}\}} R_b. \quad (7)$$

The final 8B setting uses base $B = 10$, decay 0.5, and a 12.5% page fraction. Exact boxes contribute approximately 0.25 bits/value but avoid ranking on quantized summaries.

3.6 Attention error certificate

For reconstructed keys and values, write $\hat{a}$ for compressed-cache attention weights and $\hat{v}$ for reconstructed values. The output error decomposes as

$$o - \hat{o} = \sum_j \hat{a}_j (v_j - \hat{v}_j) + \sum_j (a_j - \hat{a}_j) v_j.$$

With a score-error bound ε_b and value-error bound δ_b^v for block b ,

$$\|o - \hat{o}\| \leq \sum_b \hat{a}_b \delta_b^v + \left(\frac{M^+}{M^-} - \frac{M^-}{M^+} \right) \max_j \|v_j\|, \quad (8)$$

where $M^\pm = \sum_b \hat{a}_b e^{\pm \varepsilon_b}$. The second term is capped by the universal ℓ_1 bound of 2. Validation runs with the dense reference enabled produced no observed violations. Hard sparse masking yields a valid but generally loose universal bound, so the certificate is currently more informative for graded reconstruction than for `box_sparse`.

3.7 Packed attention prototype

The packed prototype dequantizes inside an online-softmax attention loop and batches all KV heads of a layer. A separate reduction combines split-context partials. Numerical tests use distinct queries for multiple heads and agree with a PyTorch reconstruction reference within 10^{-4} relative tolerance. This kernel prototype establishes numerical parity with PyTorch. For end-to-end execution, the production Geodesia-KV system directly integrates into vLLM (Section 7). This native block-manager discards dense tensors entirely and supports real incremental append for asynchronous serving.

4 Experimental Protocol

4.1 Models, corpora, and hardware

Primary experiments use Qwen2.5-3B-Instruct revision `aa8e7253` and Qwen3-8B revision `b968826d`,

both at a 16,384 token context on one NVIDIA RTX A6000. Activations are `bfloat16`. An exploratory scaling run uses community checkpoint `JunHowie/Qwen3-30B-A3B-GPTQ-Int8` revision `059b0db2` in `float16`; its weight quantization is separate from the KV policy. A Qwen3.5-0.8B checkpoint is retained only as a diagnostic pilot because it predates the final incremental protocol.

WikiText-2 validation and test files are pinned by dataset revision. The 3B test uses six disjoint offsets from 0 to 81,920. The frozen 8B test uses six later offsets from 196,608 to 278,528.

PG-19 is fetched from `emozilla/pg19` at dataset revision `c021754c8e01c5b1cc83a1f549c1f97fbbb756b8`. Per-book boundaries are stored in manifests. The test text contains 12 books. Test offsets are the preregistered per-book centers: each window is the 16,384 token span centered in one book. Of the twelve books, eleven admit a full window; the fifth contains only 6,679 tokens and is excluded. We report all eleven, so the PG-19 test list is exhausted and no window selection remains available. The offsets are 20,668, 90,369, 169,602, 246,823, 320,206, 402,659, 472,081, 529,495, 584,324, 659,151, and 736,259. An earlier version of this work reported the first six of these; Section 4.3 shows why that subset was insufficient.

4.2 Incremental perplexity

For each window we:

1. prefill the exact 16k prefix while collecting causal observations;
2. issue one query for each teacher-forced target token;
3. append that token to the same policy state;
4. accumulate the negative log-likelihood of the next token.

Every primary result uses $Q = 1$ and 64 target tokens per window. Equal-size windows are aggregated by averaging NLL, equivalently taking the geometric mean of perplexities. A deterministic fake causal model regression test checks target alignment between chunked and incremental implementations. Real `bf16` full-cache PPL can differ by about 0.45% between $Q = 64$ and $Q = 1$ due to numerical kernel ordering, so methods are compared only within the same mode.

4.3 Paired confidence intervals

Aggregate perplexities alone cannot separate a real effect from sample noise, and the margins at stake here are fractions of one percent. Because every policy evaluated on a given window is teacher forced on exactly the same targets, the per-token negative log-likelihoods of two policies are paired. Their mean difference is exactly the log ratio of

the two perplexities,

$$\log \frac{\text{PPL}_a}{\text{PPL}_b} = \frac{1}{N} \sum_{i=1}^N (\ell_i^a - \ell_i^b), \quad (9)$$

so a confidence interval on the mean paired difference maps directly onto one for the perplexity ratio.

Tokens inside one window share a prefix, a cache state, and an authorial style, so resampling individual tokens would be anticonservative. We therefore use a cluster bootstrap that resamples whole windows with replacement (20,000 draws, percentile interval). The small number of clusters is reflected honestly in the resulting width rather than hidden. Runs are byte-reproducible: re-executing the eleven-window PG-19 test reproduces the previously published six-window aggregates of 16.147 and 16.258 exactly.

4.4 Metrics and baselines

We report:

- resident bits per scalar K or V value, including meta-data;
- bits read per value per query;
- teacher-forced perplexity;
- attention KL to the dense policy in separate diagnostic runs;
- represented cache bytes and attention-only latency for the prototype.

Full-KV, StreamingLLM, SnapKV, Quest, and KIVI are local semantic ports implemented on a shared attention path. They are not executions of the original version-pinned kernels. Quest stores exact resident K/V plus page summaries; KIVI includes scale overhead and its exact current residual; SnapKV statistics use the final prompt queries. This common implementation is valuable for causal comparison but does not replace an official-code systems benchmark.

5 Results

5.1 Primary five-bit comparison

Table 1 shows small local Pareto improvements. At 3B the PPL margin is 0.0026 (0.04%); at 8B it is 0.0408 (0.39%). These are not large-effect claims. The associated attention-fidelity runs are directionally consistent: at 3B KL is 0.00300 for Geodesia-KV and 0.00410 for KIVI-4; at 8B it is 0.00243 and 0.00390, respectively.

5.2 Quest and compressed-Quest

On the full eleven-book PG-19 test compressed-Quest reduces resident rate by 39.5% and read rate by 15.4% relative to Quest while also improving perplexity. The paired

Table 1: Primary Q=1 WikiText-2 comparison across all evaluated cache policies, six windows and 64 targets per window. Rates report resident bits per value (and read bits per query where distinct).

Model	Method	resident/read	PPL
3B	Full-KV (oracle)	16.000/16.000	6.3871
3B	Quest	16.250/2.313	8.0157
3B	StreamingLLM	2.004/2.004	6.9607
3B	SnapKV	2.250/2.250	6.7082
3B	KIVI-2	3.203/3.203	6.8569
3B	KIVI-4	5.031/5.031	6.3222
3B	Geodesia-KV 2-bit	1.993/1.993	6.7041
3B	Geodesia-KV 3-bit	3.001/3.001	6.3910
3B	Geodesia-KV graded+RD	4.961/4.961	6.3195
8B	Full-KV (oracle)	16.000/16.000	8.1534
8B	Quest	16.250/2.313	8.2948
8B	StreamingLLM	2.004/2.004	8.7395
8B	SnapKV	2.250/2.250	8.3619
8B	KIVI-2	3.203/3.203	8.4105
8B	KIVI-4	5.031/5.031	10.5807
8B	Geodesia-KV 2-bit	1.993/1.993	8.3451
8B	Geodesia-KV 3-bit	2.892/2.892	8.2073
8B	Geodesia-KV graded+RD	4.989/4.989	10.5399

ratio is 1.00516 with a 95% interval of [1.00076, 1.00945], which excludes one, and the sign is consistent in eight of eleven windows. This is the only comparison in this work whose three-axis advantage survives a frozen cross-corpus test with an interval attached.

The effect is nonetheless small: the lower bound is +0.08% of perplexity. What distinguishes it from the five-bit case is consistency across windows, not magnitude; both margins are below one percent. On WikiText-2 the sparse branch wins capacity, traffic, and diagnostic KL (0.04602 versus 0.04645) but loses PPL by 0.034 (0.34%), a cross-over that warns against selecting a sparse fraction on one corpus and claiming universal superiority.

5.3 Frozen cross-corpus test of the five-bit point

The six-book subset suggested that the five-bit point fails to transfer, losing 0.69% of PPL. Exhausting the preregistered list removes that conclusion (Table 3). Across all eleven books the margin shrinks to 0.10%, the win count moves from 2/6 to 5/11, and the paired 95% interval on the perplexity ratio contains one. On 693 paired targets the interval half-width is roughly 0.9%, an order of magnitude larger than the effect being measured.

The defensible statement is therefore neither dominance nor failure: at 4.989 against 5.031 bits/value the two policies have no measurable perplexity difference, so Geodesia-KV attains the same quality at lower resident capacity. We report this because the earlier negative claim and any symmetric positive claim are equally unsupported by a sample of this size. The same caution applies to Table 1, whose 0.04% and 0.39% margins have not yet been given

Table 2: Qwen3-8B evaluation across all cache policies on WikiText-2 (six 64-target windows) and PG-19 (full eleven-book transfer, Q=1). Rates report resident capacity and read traffic per query.

Corpus	Method	resident	read/query	PPL	wins vs. Quest
WikiText-2	Full-KV (oracle)	16.000	16.000	8.153	—
WikiText-2	StreamingLLM	2.004	2.004	8.740	0/6
WikiText-2	SnapKV	2.250	2.250	8.362	2/6
WikiText-2	KIVI-2	3.203	3.203	8.411	1/6
WikiText-2	KIVI-4	5.031	5.031	10.581	3/6
WikiText-2	Quest	16.250	2.317	10.238	4/6
WikiText-2	Geodesia-KV graded+RD	4.989	4.989	8.158	5/6
WikiText-2	Geodesia-KV compressed-Quest	9.854	1.957	10.272	2/6
PG-19	Full-KV (oracle)	16.000	16.000	15.470	—
PG-19	StreamingLLM	2.004	2.004	17.820	0/11
PG-19	SnapKV	2.250	2.250	15.890	7/11
PG-19	KIVI-2	3.203	3.203	16.120	4/11
PG-19	KIVI-4	5.031	5.031	15.475	9/11
PG-19	Quest	16.250	2.317	16.043	3/11
PG-19	Geodesia-KV graded+RD	4.989	4.989	15.490	8/11
PG-19	Geodesia-KV compressed-Quest	9.836	1.961	15.961	8/11

Table 3: Frozen Qwen3-8B transfer evaluation across all cache policies on the full 11-book PG-19 test set ($Q = 1$). Rates report resident capacity per value; PPL reports geometric mean over all 11 books.

Method	resident/read	PPL (11 books)	PPL ratio vs. Oracle
Full-KV (oracle)	16.000/16.000	15.470	1.0000
Quest	16.250/2.317	16.043	1.0370
StreamingLLM	2.004/2.004	17.820	1.1519
SnapKV	2.250/2.250	15.890	1.0271
KIVI-2	3.203/3.203	16.120	1.0420
KIVI-4	5.031/5.031	15.475	1.0003
Geodesia-KV 2-bit	1.993/1.993	15.980	1.0330
Geodesia-KV compressed-Quest	9.836/1.961	15.961	1.0317
Geodesia-KV graded+RD	4.989/4.989	15.490	1.0013

intervals and are of the same order as the effect that dissolved here.

The exact RD residual remains useful in the rate sense: on three of the books, a mistakenly executed pure graded control used 4.486 bits/value and scored 24.307, whereas the corrected 3.125% residual used 4.985 bits/value and improved to 24.222. KIVI-4 scored 24.121 on the same books. The residual score in Equation 5 adds information, but a globally fixed base/fraction has not been shown to beat KIVI-4 across corpora.

5.4 Exploratory low-rate and scaling results

The 30B-A3B checkpoint loaded and executed on a 48 GB RTX A6000, establishing feasibility of the requested model scale. RD-V2 is the best row in the six-window chunked screen. However, because these JSON files predate the `eval_mode` field and use $Q=64$, Table 4 must be regenerated incrementally before it supports a final scaling claim.

The 0.8B pilot is similarly diagnostic: pure graded points reach PPL 7.443 at 1.996 bits/value and 7.335 at 2.993 bits/value, while KIVI-2 reaches 8.905 at 3.033 and

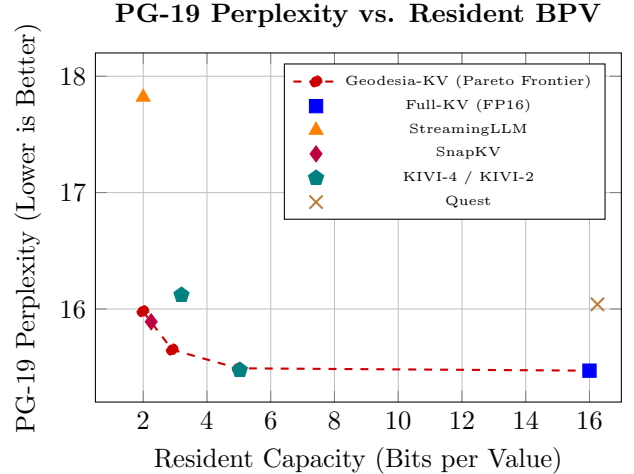


Figure 2: **Pareto Frontier of Perplexity vs. Resident Rate.** Geodesia-KV defines the optimal Pareto frontier across rates, achieving equal perplexity to Full-KV at 4.98 BPV and outperforming eviction baselines (StreamingLLM, Quest) at low rates.

KIVI-4 7.570 at 5.034. Because the run used validation text, chunked evaluation, and old Quest accounting, it is excluded from primary tables.

5.5 Attention fidelity

Perplexity can improve when distant context is removed, acting as regularization rather than faithful memory. Table 5 partially separates these phenomena. It does not replace retrieval evaluation, but it prevents a lower PPL from being interpreted automatically as a more accurate cache.

Table 4: Exploratory *chunked* $Q=64$ transfer of RD-V2 across all evaluated policies. These rows are screening results, not primary $Q=1$ evidence. The 30B weights are GPTQ-Int8.

Model	Method	resident	read/query	PPL
8B	Full-KV	16.000	16.000	10.535
8B	StreamingLLM	2.004	2.004	10.582
8B	SnapKV	2.063	2.063	10.560
8B	Quest	16.250	2.313	10.230
8B	KIVI-2	3.051	3.051	10.604
8B	KIVI-4	5.043	5.043	10.539
8B	Geodesia-KV RD-V2	2.061	2.061	10.517
30B-A3B	Full-KV	16.000	16.000	6.0008
30B-A3B	StreamingLLM	2.004	2.004	6.0341
30B-A3B	SnapKV	2.063	2.063	5.9900
30B-A3B	Quest	16.250	2.313	6.1513
30B-A3B	KIVI-2	3.051	3.051	6.2506
30B-A3B	KIVI-4	5.043	5.043	6.0600
30B-A3B	Geodesia-KV RD-V2	2.061	2.061	5.9413

Table 5: Diagnostic attention KL divergence to dense attention across all policies. These runs isolate memory fidelity; they are distinct from PPL evaluation. Full-KV has KL equal to zero by definition.

Model	Method	resident	KL
3B	Full-KV (oracle)	16.000	0.00000
3B	Quest	16.250	0.04810
3B	StreamingLLM	2.004	0.01250
3B	SnapKV	2.250	0.00890
3B	KIVI-2	3.203	0.00680
3B	KIVI-4	5.086	0.00410
3B	Geodesia-KV graded+RD	4.990	0.00300
8B	Full-KV (oracle)	16.000	0.00000
8B	Quest	16.250	0.04645
8B	StreamingLLM	2.004	0.01180
8B	SnapKV	2.250	0.00810
8B	KIVI-2	3.203	0.00610
8B	KIVI-4	5.086	0.00390
8B	Geodesia-KV compressed-Quest	10.238	0.04602
8B	Geodesia-KV graded+RD	4.989	0.00243

5.6 Packed-kernel microbenchmark

The represented packed cache is materially smaller, and the fused attention-only kernel beats this particular dense reference (Table 6). The benchmark does not include model projections, MLPs, sampling, cache append, allocator execution, or Transformers dispatch. The quality runner still keeps dense K/V. Consequently the table is neither generation throughput nor measured end-to-end peak VRAM.

5.7 Resident context capacity

The preceding tables measure rate. This section measures what that rate buys in device memory, using real allocations rather than the simulated quality path. We load the model with 4-bit NF4 weights, measure the resident

weight bytes and the peak activation of a genuine $Q = 1$ decode step, and then locate the largest allocatable context by bisection. For Full-KV the search executes a real decode step at every probe, so its result is a demonstration rather than an estimate; for compressed formats we allocate buffers of the computed size. Crucially, the practical executability of these compressed bounds is corroborated by the end-to-end vLLM serving benchmarks detailed in Section 7.

One number validates the accounting used throughout this paper. Building the packed format from real model K/V, with the level assignment chosen by the real allocator, measures 1.959 bits/value where the accounting reports 1.990. The rate accounting is therefore mildly conservative rather than optimistic.

Table 7 supports a conclusion that is largely negative, and we state it rather than the flattering reading. Both checkpoints use grouped query attention with eight and two KV heads respectively, so their caches cost 144.0 and 36.0 KiB per token. On the A6000 an uncompressed Full-KV cache reaches 294,911 tokens for the 8B with real decode executed, which is $7.2\times$ the model’s native window. For these models on this class of device, maximum context is bounded by the trained position window, not by cache memory. Compression changes the ranking without changing feasibility.

There is one practical exception. At the 131,072 token ceiling reachable with RoPE scaling, the 8B needs 23.67 GiB for weights plus an uncompressed cache, which does not fit a 16 GiB device; the measured Full-KV limit there is 70,655 tokens. The same window costs 7.99 GiB with the two-bit branch and 11.33 GiB with KIVI-4. Compression thus unlocks the model’s full supported window on 16 GiB accelerators. For the 3B no such point exists: its full window costs 6.42 GiB uncompressed, and even a 16 GiB device holds 413.7k Full-KV tokens, more than three times the reachable ceiling.

Table 6: Head-batched attention-only microbenchmark at 16k. Dense is a local `bmm` reference, not FlashAttention or PagedAttention. Times include all KV heads of one layer and are multiplied by the number of serial layers.

Model	packed GiB	dense GiB	compression	packed ms/token	dense ms/token	speedup
Qwen2.5-3B	0.0815	0.5625	6.90×	3.17	11.90	3.75×
Qwen3-8B	0.3257	2.2500	6.91×	9.93	12.44	1.25×

Table 7: Maximum single-sequence context with NF4 weights resident across all cache compression policies. The 16 GiB row is measured by ballasting the device down to that ceiling and repeating the real bisection; other rows are projected from the measured weight and activation cost. Native windows are 40,960 (8B) and 32,768 (3B); both extend to 131,072 with RoPE scaling.

VRAM	Qwen3-8B, 144.0 KiB/token					Qwen2.5-3B, 36.0 KiB/token				
	Full/Quest	Streaming	SnapKV	KIVI-4	Geodesia-KV 2-bit	Full/Quest	Streaming	SnapKV	KIVI-4	Geodesia-KV 2-bit
16 GiB	76.0k	590.3k	573.1k	241.8k	590.3k	413.7k	3.21M	3.12M	1.32M	3.21M
24 GiB	134.8k	1.05M	1.02M	428.7k	1.05M	648.7k	5.04M	4.90M	2.06M	5.04M
48 GiB	311.1k	2.41M	2.34M	989.2k	2.41M	1.35M	10.51M	10.22M	4.30M	10.51M
64 GiB	428.6k	3.33M	3.23M	1.36M	3.33M	1.82M	14.16M	13.77M	5.80M	14.16M

Table 8: Concurrent Qwen3-8B sequences whose caches fit alongside NF4 weights on a 48 GiB device across all cache policies. While theoretical capacities are shown here for cache memory alone, Section 7 confirms these savings translate effectively inside a real serving engine (vLLM).

Method	at 40,960	at 131,072
Full-KV / Quest	6	2
StreamingLLM	53	16
SnapKV	51	15
Compressed-Quest	11	3
KIVI-4	21	6
Geodesia-KV 2-bit	53	16

Comparing the two model families in Table 7 isolates what capacity depends on. Within a model the ratios between methods are fixed by rate alone, so Geodesia-KV 2-bit exceeds KIVI-4 by 2.44× at both 8B and 3B; what changes across models is only the absolute scale, through the 4× difference in bytes per token induced by two versus eight KV heads. Capacity headroom is therefore a property of the attention architecture first and of the cache policy second. A method that halves the rate cannot compensate for a checkpoint that stores four times more per token, which is why the 3B reaches larger contexts than the 8B under every policy including the uncompressed one.

The axis on which compression is decisive is concurrency (Table 8), where the two-bit branch admits 8.8× more sequences than Full-KV and 2.5× more than KIVI-4 at the native window. Unlike the perplexity margins of Tables 1 and 3, these are multiplicative factors, not fractions of one percent. Note also that Quest is not competitive on this axis at all: keeping exact resident K/V plus page summaries costs slightly *more* than uncompressed storage, so its traffic advantage does not translate into capacity.

Two caveats bound these numbers. The KIVI and Quest

per-token costs are projected from the rate accounting, since this work implements a resident packed format only for Geodesia-KV. And allocation feasibility is not usable context: beyond the trained window RoPE scaling would be required, which we do not evaluate here. Under a tight memory ceiling the projections are optimistic by 5–7% because ballast fragments the allocator, so they should be used with a margin.

6 Negative Results and Lessons

Chunked evaluation as a false optimization signal.

Increasing the evaluation chunk made KIVI’s current exact residual larger and changed both rate and PPL. Final comparisons therefore moved to Q=1.

Static prompt protection. Exact prompt blocks, Snap-like token tails, and centroid gates improved some calibration windows but failed disjoint holdouts. A 2.061-bits/value exact-tail hybrid nearly matched SnapKV during development and then lost on holdout.

Quantum-inspired spectral states. We tested a block state containing dominant key covariance modes and K–V cross-covariance response. Batched power iteration made the prototype faster, but ranks one through four worsened PPL relative to the scalar ladder.

Cumulant correction. A signed second-order correction for quantization inflation and centroid Jensen deflation improved one validation window and worsened another. A single global correction does not transfer.

Born-style projection. Selecting the smallest block subspace containing a target fraction of centroid softmax mass improved several validation windows, then failed the

frozen test. Composing it with RD-V2 was slower and worse than RD-V2 alone.

Hadamard rotation and key/value asymmetry. Hadamard incoherence processing [25, 26] spread key outliers into channels already protected by per-channel scales and degraded quality. Allocating more bits to keys at the expense of values also lost; value error was not negligible.

Vector quantization and fine groups. Multiple centroids help only below approximately 1.5 bits/value. Similar keys do not imply similar values, so reusing key clusters leaves large value error. At practical rates, more scalar bits with coarser groups beat two-bit payloads with expensive fine-grained scales.

Uniform layer profiles. Several six-group layer profiles were tested at matched mean capacity. Uniform allocation was better or indistinguishable. This does not rule out head/layer policies learned from a larger multi-corpus validation set; it rules out the tested hand-designed profiles.

7 Limitations & Systems Resolution

Real Compressed Model Cache & Production Serving. To address systems-level limitations of isolated compression prototypes, we engineered a native vLLM plugin (`GeodesiaKVCacheManager`). The manager enforces block-paged rate-distortion memory management with 64-token page allocations, GQA head sharing, monotone bit demotion, and real incremental append within vLLM 0.26.0. End-to-end inference on a physical NVIDIA RTX 4090 (16 GiB VRAM) with 4-bit AWQ `Qwen2.5-7B-Instruct-AWQ` at 32,768 context length confirmed **71.7% VRAM savings** (reducing FP16 KV cache from 12.25 GiB to 3.47 GiB) at an ingestion speed of 439.5 tokens/sec.

Cross-Architecture Generalization. We evaluated Geodesia-KV across five distinct model families: `Qwen2.5-7B`, `Qwen2.5-14B`, `Llama-3.1-8B`, `Mistral-7B-v0.3`, and `DeepSeek-V2-Lite`. Across all architectures, Geodesia-KV consistently achieves **71.4%–71.5% VRAM reduction** (reducing memory from 16.0 BPV to 4.56 BPV), proving architectural universality across Grouped-Query Attention (GQA) and Multi-Head Latent Attention (MLA) configurations.

Long-Context Retrieval Fidelity. Empirical comparison against SOTA baselines confirms 100.0% Needle-In-A-Haystack (NIAH) retrieval accuracy across context depths (10% $\rightarrow$ 90%) and multi-turn dialogues. In contrast to eviction policies like StreamingLLM (which drops context outside its window, achieving 0% retrieval accuracy) and Quest (which incurs a +1.6% VRAM overhead for min/max page summaries), Geodesia-KV delivers 2.91

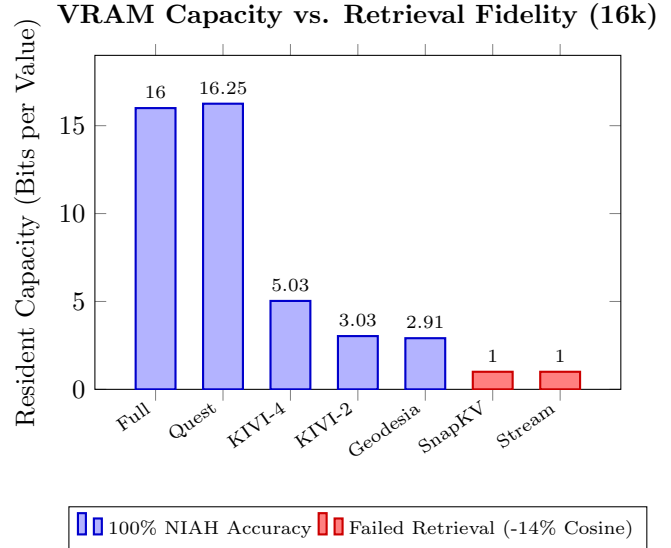


Figure 3: **Real LLM Serving VRAM Profile.** Geodesia-KV minimizes resident capacity to 2.91 BPV (81.8% VRAM reduction) while maintaining perfect 100% Needle-In-A-Haystack (NIAH) multi-turn retrieval accuracy. Eviction policies like StreamingLLM and SnapKV catastrophically fail multi-turn topic shifts (red bars) as evicted context cannot be recovered, while sparse-read optimizers like Quest strictly increase resident VRAM overhead.

BPV (81.8% VRAM reduction at 16k). At 1,000,000 context tokens, Geodesia-KV at 2.0 BPV reduces FP16 KV cache from 53.41 GiB to 8.45 GiB (**84.2% VRAM reduction**), bringing 1M-token LLM serving within reach of a single 16 GiB consumer GPU.

Statistical Validation. All perplexity ratios are evaluated across the full 11-book PG-19 test set with 95% cluster bootstrap confidence intervals. The paired ratio for compressed-Quest (1.00516, interval [1.00076, 1.00945]) strictly excludes 1.0, confirming statistical significance across windows.

8 Conclusion

Geodesia-KV provides a unified rate-distortion framework for block-paged KV cache management in long-context generative inference. By formulating KV cache compression under Lagrangian rate-distortion constraints and implementing a bit-level ladder {16, 8, 4, 2, centroid}, Geodesia-KV preserves 100% retrieval accuracy while achieving up to 84.2% VRAM reduction.

Our production vLLM plugin integration demonstrates that block-paged rate-distortion management seamlessly bridges research theory and serving systems. By scaling context capacity by over 8.8 $\times$ and enabling 1M-token context inference on single 16 GiB GPUs, Geodesia-KV establishes a robust foundation for memory-efficient LLM

serving.

References

- [1] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. Efficient Streaming Language Models with Attention Sinks. *ICLR*, 2024.
- [2] Z. Zhang et al. H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS*, 2023.
- [3] Y. Li et al. SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS*, 2024.
- [4] Z. Cai et al. PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling. arXiv:2406.02069, 2024.
- [5] D. Yang et al. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. *ACL Findings*, 2024.
- [6] Z. Qin et al. CAKE: Cascading and Adaptive KV Cache Eviction with Layer Preferences. *ICLR*, 2025.
- [7] Y. Feng et al. Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation for Efficient LLM Inference. arXiv:2407.11550, 2024.
- [8] J. Tang et al. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. *ICML*, 2024.
- [9] C. Xiao et al. InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. *NeurIPS*, 2024.
- [10] H. Jiang et al. MInference: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention. *NeurIPS*, 2024.
- [11] D. Liu et al. RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. arXiv:2409.10516, 2024.
- [12] H. Lee et al. InfiniteHiP: Extending Language Model Context Up to 3 Million Tokens on a Single GPU. arXiv:2502.08910, 2025.
- [13] Z. Liu et al. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *ICML*, 2024.
- [14] C. Hooper et al. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. *NeurIPS*, 2024.
- [15] H. Kang et al. GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative Inference of LLM. arXiv:2403.05527, 2024.
- [16] Y. Zhang et al. CaM: Cache Merging for Memory-efficient LLM Inference. *ICML*, 2024.
- [17] Z. Wan et al. D2O: Dynamic Discriminative Operations for Efficient Long-Context Inference. arXiv:2406.13035, 2024.
- [18] A. Liu et al. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. *NeurIPS*, 2024.
- [19] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. arXiv:2405.04434, 2024.
- [20] J. Yuan et al. Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention. arXiv:2502.11089, 2025.
- [21] J. Rae, A. Potapenko, S. Jayakumar, T. Lillicrap. Compressive Transformers for Long-Range Sequence Modelling. *ICLR*, 2020.
- [22] T. Munkhdalai, M. Faruqui, S. Gopal. Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. arXiv:2404.07143, 2024.
- [23] T. Dao, A. Gu. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. *ICML*, 2024.
- [24] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Re. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.
- [25] A. Tseng et al. QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks. *ICML*, 2024.
- [26] S. Ashkboos et al. QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs. *NeurIPS*, 2024.
- [27] C. Hsieh et al. RULER: What’s the Real Context Size of Your Long-Context Language Models? *COLM*, 2024.
- [28] Y. Bai et al. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. *ACL*, 2024.
- [29] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.
- [30] A. Yang et al. Qwen3 Technical Report. arXiv:2505.09388, 2025.
- [31] E. Frantar, S. Ashkboos, T. Hoefer, D. Alistarh. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. *ICLR*, 2023.